

Sanjeevni: Engineering Robust Safety Guardrails and Deterministic Fallbacks in LLM-Based Rural Telehealth

Technical Project & Safety Case Study

Candidate Submission for the Singapore AI Safety Fellowship

Abstract

*Deploying Large Language Models (LLMs) in safety-critical medical contexts exposes users to severe risks, including diagnostic hallucinations, dosage inaccuracies, language alignment drift, and adversarial prompt injections (jailbreaking). This paper presents **Sanjeevni**, an open-source rural telehealth platform, as a case study in engineering robust, defense-in-depth safety boundaries around generative AI models. We describe a layered, dual-system architecture combining deterministic input pre-filtering, structured API-level schema enforcement, real-time multilingual script validation, and automated deterministic fallbacks. By intercepting safety-violating queries before LLM invocation, enforcing strict Zod validation schemas on LLM outputs, and maintaining a strict Human-in-the-Loop (HITL) doctor-approval model, Sanjeevni demonstrates a practical methodology for deploying generative AI safely and reliably in resource-constrained, high-stakes environments.*

1. Introduction

Expanding clinical access to underserved rural regions is one of the most promising applications of conversational AI. In these environments, patients face language barriers, low literacy rates, and a scarcity of medical professionals. LLMs can bridge this gap by performing pre-triage symptom assessments, organizing scheduling, and translating medical jargon into intuitive, regional dialects.

However, LLMs are fundamentally non-deterministic and prone to confabulations, which can be catastrophic in medicine. An unaligned LLM could provide invalid self-diagnoses, prescribe incorrect chemical dosages, or fail to recognize critical emergencies. AI safety in clinical technology requires absolute safety bounds: the LLM must never act as an autonomous doctor, it must resist attempts by users to bypass system prompts, and it must degrade to a safe state if the model output is malformed. This case study details the concrete engineering implementations built into Sanjeevni to guarantee these constraints.

2. Dual-System Safety Architecture

Sanjeevni uses a multi-tier, defense-in-depth pipeline written in TypeScript (Hono and Next.js) and backed by PostgreSQL with Prisma ORM. Rather than relying on simple prompt engineering (which is highly vulnerable to jailbreak overrides), Sanjeevni encapsulates the generative LLM (Google Gemini) within a rigid wrapper composed of three layers:

- **Layer 1: Deterministic Pre-Filtering (Input):** Evaluates user requests using rule-based algorithms to intercept emergency situations and malicious injections before any API token is spent.
- **Layer 2: Constrained Structured Generation (Model):** Utilizes strict API-level JSON schemas and system rules to guide the LLM's output shape and bounds.
- **Layer 3: Schema and Language Alignment Verification (Output):** Validates the generated JSON using Zod, checks native script layout, and triggers deterministic fallbacks upon failure.

3. Pre-Processing Guardrails: Input Sanitization

To prevent prompt injection attacks (such as instructions to 'ignore all previous safety guidelines' or requests to synthesize medication dosages), the server passes inputs through a local validator first. This includes:

- **Emergency Short-Circuiting:** A regex scan matches critical emergency terms (e.g., 'chest pain', 'breathing difficulty', 'unconscious', 'severe bleeding'). If triggered, the system immediately returns a predefined response advising the user to contact emergency services (112 in India) and flags the admin dashboard, completely bypassing the LLM.
- **Safety Refusal Checks:** Detects queries demanding autonomous diagnosis or dosage numbers (e.g., 'prescribe', 'mg dosage', 'diagnose me'). If detected, the API rejects the request with a standard refusal message.

Example of the rule-based safety pre-filter implemented in Sanjeevni's backend:

```
// Pre-processing safety check inside Hono's route handler
function ruleBasedEmergencyCheck(input) {
  const combined = `${input.symptoms} ${input.context || ''}`.toLowerCase();
  const matched = emergencyKeywords.find(k => combined.includes(k));
  if (matched) return matched;

  // Detect low oxygen emergency bounds
  const oxygenMatch = combined.match(/(?:spo2|oxygen)\\s*(\\d+)/i);
  if (oxygenMatch && Number(oxygenMatch[1]) < 90) {
    return "oxygen level below 90%";
  }
  return null;
}
```

4. Output Schema Enforcement via Zod

If a request passes the input filters, it is sent to the LLM. In addition to a specialized system prompt restricting its role to triage or prescription simplification, Sanjeevni configures the API to return a strict JSON block. Once received, the backend validates the JSON structure using Zod schemas to ensure type and field safety before returning it to the user.

The symptom triage response schema is configured as follows:

```
const symptomCheckResponseSchema = z.object({
  summary: z.string().min(5).max(600),
  triageLevel: z.enum(["RED", "YELLOW", "GREEN", "BLUE"]),
  explanation: z.string().min(5).max(1000),
  recommendedAction: z.string().min(5).max(600),
  disclaimer: z.string().min(10).max(400),
}).strict();
```

If the LLM's response fails validation—due to malformed JSON, out-of-bounds strings, or hallucinated enum values—the system catches the error and executes a deterministic fallback generator. This fallback creates a safe, standardized message, preventing the client app from rendering unpredictable or broken layouts.

5. Multilingual Script Verification

A major problem with multilingual LLM deployments is language drift and transliteration failure. If a user asks for Hindi instructions, the LLM might return Hindi sentences written in Latin characters (Hinglish), which is difficult for rural patients to read, or drift back to English entirely. Sanjeevni solves this by implementing script verification:

- **Unicode Block Verification:** Maps requested languages to native Unicode ranges (e.g., Devanagari [\u0900-\u097F] for Hindi, Tamil [\u0B80-\u0BFF], Bengali [\u0980-\u09FF]).
- **Alignment Check:** Compares the length of characters in the requested script against Latin alphabet characters. If the target script is absent or fails validation, the output is rejected.
- **Fallback Translator Call:** On alignment failure, the backend attempts to re-translate the raw text via a dedicated, single-purpose translation prompt. If that also fails, it reverts to a local library of static translations.

6. Human-in-the-Loop Alignment & Cryptographic Verification

The primary safety principle of Sanjeevni is that autonomous AI is never permitted to make critical medical decisions. The AI acts as an administrative assistant, while humans (licensed medical professionals) make all final clinical decisions. This boundary is enforced through:

- **Separation of Concerns:** The symptom checker only provides triage recommendations for scheduling, not diagnostic judgments. Prescription generation requires a licensed doctor's signature in the database.
- **Prescription QR Verification:** When a doctor writes a prescription, the system generates a secure, unique UUID (the qrToken). Patients receive a visual schedule translation, but pharmacies only dispense medications after scanning the QR token and loading the official, unmodifiable prescription directly from the database server. This

prevents any client-side tampering or injection of fake medicines.

7. Comprehensive Audit Logging

For post-incident analysis and alignment monitoring, Sanjeevni records all AI transactions in an AuditLog model. This records the user ID, action type (e.g., AI_SYMPTOM_TRIAGE, AI_PRESCRIPTION_SIMPLIFY), the triage levels determined by the AI, and metadata. This structured audit trail allows engineers to evaluate hallucination rates, optimize prompts, and identify malicious users attempting to break system boundaries.

8. Significance to AI Safety Research

Sanjeevni provides a concrete, real-world prototype for AI alignment and safety in medicine. While theoretical research focuses on reward-modeling and reinforcement learning (RLHF), practical applications require runtime validation. Sanjeevni's dual-system architecture shows how safety-critical systems can neutralize the stochastic vulnerabilities of deep learning models by wrapping them in deterministic boundaries. This defense-in-depth approach is directly applicable to broader research in LLM agent alignment, multi-agent consensus, and automated safety evaluation in high-stakes environments.